

Periféricos y Dispositivos de Interfaz Humana

Universidad de Granada - Curso 2025/2026

Trabajo: Cartel de farmacia con Arduino

Por Alejandro López Jiménez

Índice

1. Introducción	2
2. Objetivo	2
3. Componentes hardware	3
4. Implementación	6
5. Código	9
6. Puesta en escena	18
7. Conclusiones	18

1. Introducción

Este trabajo consiste en la implementación de un cartel de farmacia usando Arduino por lo que lo adecuado sería primero definir qué es un cartel de farmacia y qué es Arduino.

¿Qué es un cartel de farmacia?

Un cartel de farmacia consiste en un letrero digital destinado a anunciar la presencia de una farmacia que se suele situar en la fachada de estas.

Estos carteles suelen funcionar continuamente en horario diurno y normalmente, a parte de mensajes publicitarios o informativos, también incluyen información adicional cotidiana como la fecha, la hora o la temperatura.

¿Qué es Arduino?

Arduino es una plataforma de creación de electrónica de código abierto, la cual permite crear diferentes tipos de microordenadores diseñando tanto el software como el hardware. El esquema básico de un circuito en Arduino consiste en una placa controladora, la cual dispone de un microcontrolador y diferentes entradas para asignación de pines, voltajes y tierra; y los diferentes periféricos que se le conectan para formar distintos circuitos.

Mediante un software, llamado Arduino IDE, se puede programar la lógica con la que actuarán dichos componentes.

2. Objetivo

El objetivo de este trabajo es implementar en Arduino una serie de utilidades que en conjunto den forma a lo que se podría considerar el funcionamiento básico de un cartel de farmacia.

Entre dichas utilidades podemos encontrar:

- Mostrar información acerca de la farmacia (nombre, horario, etc).
- Mostrar la temperatura y la humedad del ambiente.
- Mostrar la fecha y la hora en tiempo real.

Nota: la humedad no es algo que se muestre de forma habitual en los carteles de farmacia pero, dado que el sensor que lee la temperatura ofrece también este parámetro, me ha parecido interesante incluirlo.

Para conseguirlo, utilizo una placa Arduino UNO y diferentes componentes, de los que hablaré a continuación, junto con un código que define la lógica con la que actuarán.

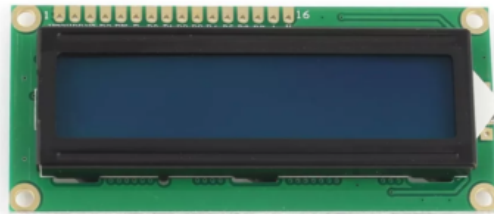
3. Componentes hardware

El elemento principal es la placa controladora UNO de Arduino, la cual se conecta con distintos componentes hardware para lograr el objetivo anterior mencionado.

Dichos componentes son los siguientes:

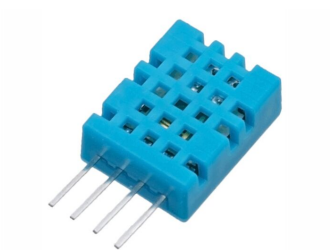
Pantalla LCD 16x2

Consiste en una pantalla de cristal líquido dividida en 32 segmentos, 2 filas x 16 columnas capaz de mostrar texto y números. Se comunica con el Arduino utilizando 6 pines (RS, E, D4-D7). El pin **V0** controla cuánto contrastan los caracteres con el fondo, conectándose al pin central de un potenciómetro. El pin **RS (Register Select)** indica si lo que enviamos es un **comando** o un **carácter** para mostrar.



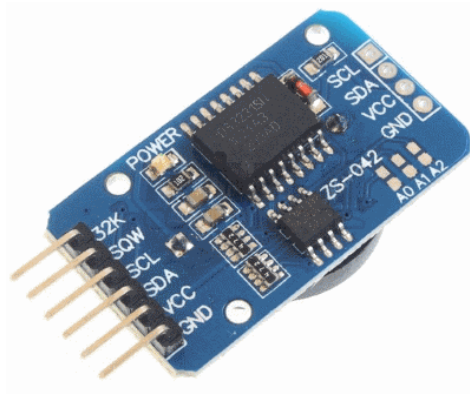
Sensor DHT11

Sensor digital capaz de medir temperatura y humedad. Se comunica con el Arduino mediante un solo cable de datos. Requiere un intervalo mínimo de 2 segundos entre lecturas consecutivas.



Sensor RTC DS3231

Reloj en tiempo real (**Real Time Clock**) que mantiene la fecha y la hora incluso cuando el Arduino está apagado gracias a una pila incorporada. Se comunica con el Arduino mediante el protocolo **I2C**, utilizando los pines **SDA** y **SCL**. El pin **SDA** (**S**erial **D**ata) transporta los datos mientras que **SCL** (**S**erial **C**lock) es el reloj de sincronización que marca el ritmo con el que se leen.



Potenciómetro de 10K Ohmios

Resistencia variable de tres líneas que se utiliza para regular el contraste de la pantalla LCD mediante su pin central (wiper).



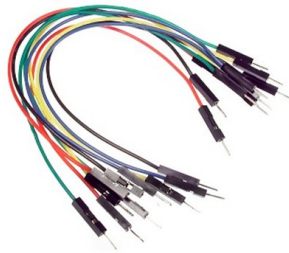
Resistencia de 10K Ohmios

Resistencia fija utilizada como pull-up en la línea de datos del sensor DHT11. Se conecta entre el pin de datos y la alimentación de 5V.



Cables de conexión

Cables tipo dupont macho-macho y macho-hembra utilizados para realizar las conexiones entre la placa Arduino, la protoboard y los distintos componentes.



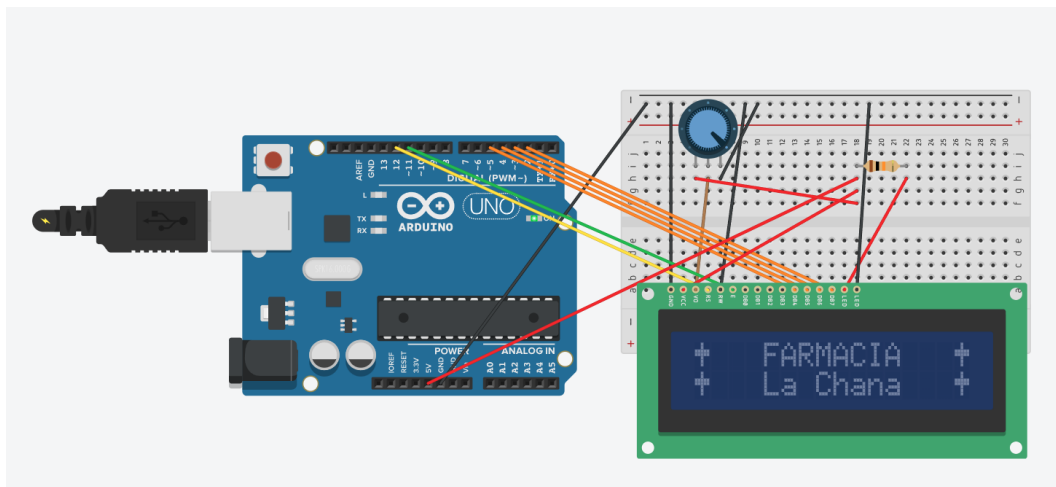
4. Implementación

Las conexiones de los diferentes componentes con el Arduino son las siguientes:

Pantalla LCD 16x2

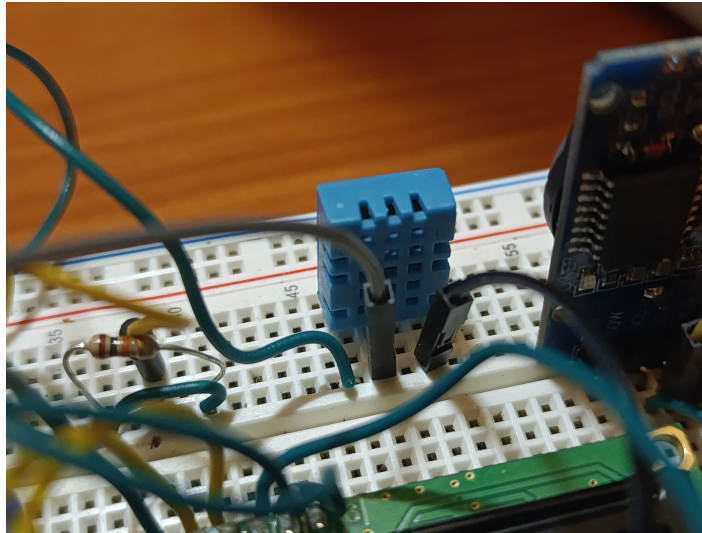
- GND $\Rightarrow$ Tierra
- VCC $\Rightarrow$ 5V
- Vd $\Rightarrow$ Pin central del potenciómetro
- RS $\Rightarrow$ Pin 12
- RW $\Rightarrow$ Tierra
- E $\Rightarrow$ Pin 11
- DB0 $\Rightarrow$ sin conectar
- DB1 $\Rightarrow$ sin conectar
- DB2 $\Rightarrow$ sin conectar
- DB3 $\Rightarrow$ sin conectar
- DB4 $\Rightarrow$ Pin 5
- DB5 $\Rightarrow$ Pin 4
- DB6 $\Rightarrow$ Pin 3
- DB7 $\Rightarrow$ Pin 2
- (A) $\Rightarrow$ 5V a través de la resistencia de 10K Ohmios
- (K) $\Rightarrow$ Tierra

Vista esquemática en TinkerCad



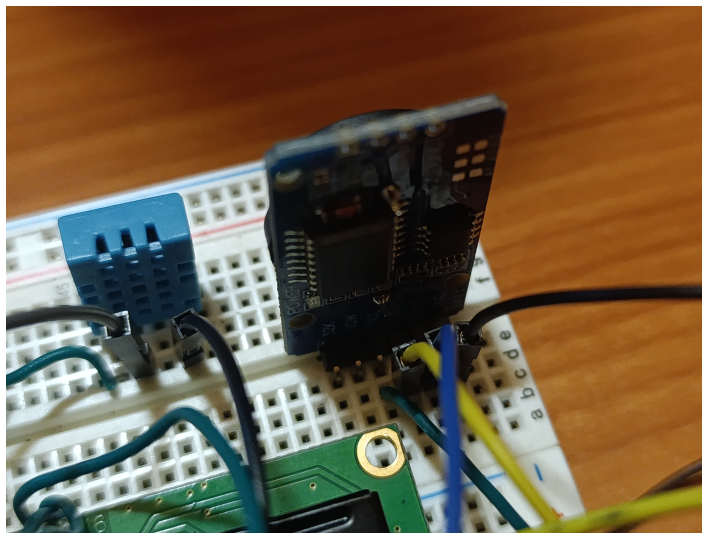
Sensor DHT11

- VCC $\Rightarrow$ 5V
- Data $\Rightarrow$ Pin 10
- GND $\Rightarrow$ Tierra



Sensor RTC DS3231

- VCC $\Rightarrow$ 5V
- GND $\Rightarrow$ Tierra
- SDA $\Rightarrow$ Pin A4
- SCL $\Rightarrow$ Pin A5



Potenciómetro de 10K Ohmios

- Línea 1 (extremo) $\Rightarrow$ 5V
- Línea 2 (central) $\Rightarrow$ Pin Vd del LCD
- Línea 3 (extremo) $\Rightarrow$ Tierra

Resistencia de 10K

- Extremo 1 $\Rightarrow$ Pin Data del sensor DHT11
- Extremo 2 $\Rightarrow$ 5V del Arduino

5. Código

El código para definir la lógica del cartel se divide en dos archivos, **ajustar_hora_rtc.ino** y **cartel_farmacia.ino**. El primero sirve para ajustar la hora del reloj la primera vez que se utiliza mientras que el segundo es el principal y define la lógica del cartel.

ajustar_hora_rtc.ino

Se sube al Arduino una sola vez para poner en hora el DS3231.

En el setup se inicia la comunicación con el reloj mediante `rtc.begin()` y le envía la fecha y hora actuales con `rtc.adjust()`, utilizando las macros `__DATE__` y `__TIME__`, que el compilador sustituye automáticamente por la fecha y hora del ordenador en el momento de compilar.

La función `loop()` muestra el resultado por el Serial Monitor del IDE para comprobar que el ajuste ha sido exitoso.

```
1 #include <Wire.h>
2 #include <RTClib.h>
3
4 RTC_DS3231 rtc;
5
6 void setup() {
7     Serial.begin(9600);
8     rtc.begin();
9     rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
10 }
11
12 void loop() {
13     DateTime ahora = rtc.now();
14
15     if (ahora.day() < 10) Serial.print('0');
16     Serial.print(ahora.day());
17     Serial.print('/');
18     if (ahora.month() < 10)
19         Serial.print('0');
20     Serial.print(ahora.month());
21     Serial.print('/');
22     Serial.print(ahora.year());
23     Serial.print(" ");
24
25     if (ahora.hour() < 10) Serial.print('0');
26     Serial.print(ahora.hour());
27     Serial.print(':');
28     if (ahora.minute() < 10) Serial.print('0');
29     Serial.print(ahora.minute());
30     Serial.print(':');
31     if (ahora.second() < 10) Serial.print('0');
32     Serial.println(ahora.second());
33
34     delay(1000);
35 }
```

cartel_farmacia.ino

Este archivo define la lógica del cartel mediante distintas partes que voy a explicar a continuación, entre las cuales encontramos:

- Declaración de librerías y asignación de pines
- Declaración de constantes y variables globales
- Definición de caracteres personalizados
- Funciones auxiliares
- Función leerSensor()
- Función mostrarHora()
- Función mostrarMensaje()
- Función setup()
- Función loop

Declaración de librerías y asignación de pines

Las librerías son necesarias ya que incluyen funciones con las que poder controlar la información recogida por los sensores, las utilizadas en este proyecto son las siguientes:

- **LiquidCrystal.h:** esta librería viene preinstalada con el IDE de Arduino y se utiliza para controlar la pantalla LCD 16x2. Incluye funciones como `lcd.begin(16, 2)` para inicializar la pantalla, `lcd.setCursor(col, fila)` para seleccionar la posición en la que escribir, `lcd.print(...)` para escribir o `lcd.write(byte)` para escribir un byte específico, utilizada para mostrar los caracteres personalizados que mencionaré más adelante, entre muchas otras.
- **DHT.h:** sirve para leer el sensor de temperatura y humedad DHT11, incluye funciones como `readTemperature()` y `readHumidity()` para medir temperatura y humedad respectivamente.
- **RTCLib.h:** sirve para leer y configurar relojes en tiempo real como el DS3231. El DS3231 guarda la hora en sus registros internos en formato BCD y esta librería se encarga de convertir entre BCD y números normales. Incluye funciones como `rtc.begin()` para inicializar la comunicación con el chip, `rtc.lostPower()` para comprobar si el reloj perdió la hora (por pila agotada o ser la primera vez que se utiliza), `rtc.adjust(DateTime)` para ajustar la hora o `rtc.now()` para generar un objeto DateTime con la hora actual.
- **Wire.h:** también viene preinstalada con el IDE y sirve para gestionar la comunicación por el bus I2C. I2C (Inter-Integrated Circuit) es un protocolo de comunicación que permite conectar varios chips usando solo dos cables. El RTC DS3231 se comunica por I2C por lo que también utiliza esta librería internamente y RTCLib.h depende de ella.

En este fragmento del código también asignamos los pines y creamos un objeto de tipo RTC_DS3231 (proporcionado por la librería RTCLib.h) para trabajar con el RTC.

```

1 #include <LiquidCrystal.h>
2 #include <DHT.h>
3 #include <Wire.h>
4 #include <RTClib.h>
5
6 // LCD: RS, E, D4, D5, D6, D7
7 LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
8
9 // DHT11 en el pin 10
10 DHT dht(10, DHT11);
11
12 // RTC DS3231 (I2C en A4/A5)
13 RTC_DS3231 rtc;

```

Definición de caracteres personalizados

El controlador HD44780 de la pantalla LCD dispone de una memoria interna llamada **CGRAM** (Character Generator RAM) donde es posible almacenar hasta **8 caracteres personalizados**. En mi trabajo la he utilizado para dibujar símbolos ilustrativos coherentes con cada mensaje mostrado en pantalla.

He definido los siguientes caracteres:

- **Cruz de farmacia:** símbolo clásico de una farmacia, dibujado en las cuatro esquinas de la pantalla.
- **Símbolo de grado:** pequeño círculo situado en la esquina superior derecha del valor de temperatura (°C).
- **Gota:** icono de una gota de agua que acompaña a la lectura de humedad.
- **Relojito:** pequeño reloj que se dibuja junto a la fecha cuando se muestra la hora actual.

El prefijo 0b indica que el número está escrito en binario, lo que permite que el dibujo del carácter sea visualmente reconocible directamente en el código: cada 1 representa un píxel encendido y cada 0 un píxel apagado.

El carácter de la cruz de farmacia, por ejemplo, se vería así:

```

0b00100 -> . . X . .
0b00100 -> . . X . .
0b11111 -> X X X X X
0b11111 -> X X X X X
0b00100 -> . . X . .
0b00100 -> . . X . .
0b00100 -> . . X . .
0b00000 -> . . . . .

```

El código es el siguiente:

```

1 // Car cter personalizado: cruz de farmacia
2 byte cruzFarmacia[8] = {
3   0b00100,
4   0b00100,

```

```

5     0b11111,
6     0b11111,
7     0b00100,
8     0b00100,
9     0b00100,
10    0b00000
11 };
12
13 // Car cter personalizado: s mbolo de grado
14 byte grado[8] = {
15     0b01100,
16     0b10010,
17     0b10010,
18     0b01100,
19     0b00000,
20     0b00000,
21     0b00000,
22     0b00000
23 };
24
25 // Car cter personalizado: gota
26 byte gota[8] = {
27     0b00100,
28     0b00100,
29     0b01010,
30     0b01010,
31     0b10001,
32     0b10001,
33     0b10001,
34     0b01110
35 };
36
37 // Car cter personalizado: relojito
38 byte reloj[8] = {
39     0b00000,
40     0b01110,
41     0b10101,
42     0b10111,
43     0b10001,
44     0b01110,
45     0b00000,
46     0b00000
47 };

```

Declaración de constantes y variables globales

Para gestionar y coordinar las distintas funciones del programa declaro una serie de constantes y variables globales que son accesibles desde cualquier punto del programa.

Según su finalidad se pueden agrupar en:

```

1 unsigned long ultimoParpadeo = 0;
2 unsigned long ultimoCambioMsg = 0;
3 unsigned long ultimaLecturaDHT = 0;
4 unsigned long ultimaActHora = 0;

```


Estas cuatro variables almacenan el último instante (medido con `millis()`) en el que se ejecutó cada una de las tareas periódicas: el parpadeo de las cruces, el cambio de mensaje, la lectura del DHT11 y la actualización del reloj. Se declaran como **unsigned long** porque `millis()` devuelve un número de milisegundos que puede llegar a ser muy grande por lo que no cabría en un `int`.

```
1 bool cruzVisible = true;
2 int mensajeActual = 0;
```

`cruzVisible` es una variable booleana que indica si las cruces deben mostrarse o estar ocultas en cada instante; se cambia cada 600 ms para producir el efecto de parpadeo. `mensajeActual` es una variable que varía entre 0 y 4 e indica cuál de los cinco mensajes se está mostrando en pantalla.

```
1 float tempC = 0.0;
2 float humedad = 0.0;
```

Estas dos variables almacenan los últimos valores válidos leídos del sensor DHT11. Se declaran como `float` para poder representar decimales, y se conservan globalmente para que la función que dibuja en pantalla pueda acceder a ellos sin tener que volver a consultar el sensor en cada ciclo.

```
1 const int INTERVALO_PARPADEO = 600;      // ms
2 const int INTERVALO_MENSAJE = 3000;      // ms
3 const int INTERVALO_LECTURA_DHT = 2500; // ms
4 const int INTERVALO_HORA = 1000;         // ms
```

Estas constantes definen en milisegundos cada cuánto tiempo debe ejecutarse cada una de las tareas periódicas. El intervalo del DHT11 se establece en 2.5 segundos por exigencia del propio sensor, que necesita un mínimo de 2 segundos entre lecturas consecutivas.

Funciones auxiliares

He definido tres funciones auxiliares que encapsulan tareas repetitivas utilizadas en distintos puntos del programa, de este modo evito la duplicación de código.

Entre dichas funciones podemos encontrar:

void dibujarCruces() Dibuja las cuatro cruces de farmacia en las esquinas de la pantalla, en las posiciones (0,0), (15,0), (0,1) y (15,1). Según el valor de la variable global `cruzVisible`, escribe el carácter personalizado correspondiente a la cruz o un espacio en blanco, produciendo así el efecto de parpadeo. Esta función es invocada cada 600 milisegundos desde el bucle principal.

```

1 void dibujarCruces() {
2     lcd.setCursor(0, 0);
3     lcd.write(cruzVisible ? (uint8_t)0 : ' ');
4     lcd.setCursor(15, 0);
5     lcd.write(cruzVisible ? (uint8_t)0 : ' ');
6     lcd.setCursor(0, 1);
7     lcd.write(cruzVisible ? (uint8_t)0 : ' ');
8     lcd.setCursor(15, 1);
9     lcd.write(cruzVisible ? (uint8_t)0 : ' ');
10 }

```

void imprimirDosDigitos(int n) Añade un 0 delante de cada dígito menor que 10. Esta función se utiliza al mostrar la fecha y la hora para garantizar que los campos mantengan siempre el mismo formato (por ejemplo, mostrando "08:05:03.^{en} lugar de "8:5:3"), lo que mejora la estética en pantalla.

```

1 void imprimirDosDigitos(int n) {
2     if (n < 10) lcd.print('0');
3     lcd.print(n);
4 }

```

void limpiarCentro() Borra el contenido de la zona central de la pantalla escribiendo espacios en blanco desde la columna 1 hasta la 14 en ambas filas, dejando intactas las columnas 0 y 15 donde se encuentran las cruces de farmacia.

Se utiliza en lugar de `lcd.clear()` para evitar que las cruces parpadeantes desaparezcan momentáneamente cada vez que cambia el mensaje.

```

1 void limpiarCentro() {
2     for (int fila = 0; fila < 2; fila++) {
3         lcd.setCursor(1, fila);
4         for (int c = 1; c <= 14; c++) lcd.print(' ');
5     }
6 }

```

Función leerSensor()

Utiliza las funciones `readTemperature()` y `readHumidity()` pertenecientes a la librería DHT.h para medir la temperatura y la humedad y almacenarlas en sus respectivas variables globales.

```

1 void leerSensor() {
2     float t = dht.readTemperature();
3     float h = dht.readHumidity();
4     if (!isnan(t)) tempC = t;
5     if (!isnan(h)) humedad = h;
6 }

```

Función mostrarHora()

Obtiene el instante actual del RTC con `rtc.now()` y lo guarda en un objeto `DateTime`. A continuación dibuja en la fila superior un icono de reloj seguido de la fecha, y en la fila inferior la hora.

Utiliza la función auxiliar `imprimirDosDigitos()` para que todos los valores mantengan un ancho fijo de dos cifras.

```
1 void mostrarHora() {
2     DateTime ahora = rtc.now();
3     limpiarCentro();
4
5     // Fila superior: icono de reloj + fecha
6     lcd.setCursor(2, 0);
7     lcd.write((uint8_t)3);
8     lcd.print(' ');
9     imprimirDosDigitos(ahora.day());
10    lcd.print('/');
11    imprimirDosDigitos(ahora.month());
12    lcd.print('/');
13    lcd.print(ahora.year());
14
15    // Fila inferior: hora centrada
16    lcd.setCursor(4, 1);
17    imprimirDosDigitos(ahora.hour());
18    lcd.print(':');
19    imprimirDosDigitos(ahora.minute());
20    lcd.print(':');
21    imprimirDosDigitos(ahora.second());
22 }
```

Función `mostrarMensaje()`

Decide qué información se muestra en la zona central del LCD según el valor de la variable `mensajeActual`, que va rotando entre 0 y 4.

Antes de pintar, llama a `limpiarCentro()` para borrar el mensaje anterior sin tocar las cruces de las esquinas.

Después, mediante un switch, escribe el nombre de la farmacia, el aviso de apertura, la hora actual, la temperatura o la humedad.

```
1 void mostrarMensaje() {
2     limpiarCentro();
3
4     switch (mensajeActual) {
5         case 0:
6             lcd.setCursor(2, 0);
7             lcd.print("  FARMACIA");
8             lcd.setCursor(2, 1);
9             lcd.print("  La Chana");
10            break;
11        case 1:
12            lcd.setCursor(2, 0);
13            lcd.print("  ABIERTO");
14            lcd.setCursor(2, 1);
15            lcd.print("  24 HORAS");
16            break;
17        case 2:
18            mostrarHora();
19            break;
20        case 3:
21            lcd.setCursor(2, 0);
```

```

22     lcd.print(" Temperatura");
23     lcd.setCursor(5, 1);
24     lcd.print(tempC, 1);
25     lcd.write((uint8_t)1); //
26     lcd.print("C");
27     break;
28 case 4:
29     lcd.setCursor(2, 0);
30     lcd.print(" Humedad ");
31     lcd.write((uint8_t)2); // gota
32     lcd.setCursor(5, 1);
33     lcd.print(humedad, 0);
34     lcd.print(" %");
35     break;
36 }
37 }

```

Función setup()

Se ejecuta al comienzo del programa, antes del bucle principal. Sus funciones son las siguientes:

- Inicializa la pantalla LCD con `lcd.begin()`
- Carga los cuatro caracteres personalizados en la memoria CGRAM mediante `createChar()`.
- Arranca el sensor DHT11 y el reloj DS3231.
- Si el RTC ha perdido la hora (por la pila), la reajusta automáticamente a la hora de compilación.
- Finalmente muestra el mensaje de bienvenida durante 1.5 segundos.

```

1 void setup() {
2     lcd.begin(16, 2);
3     lcd.createChar(0, cruzFarmacia);
4     lcd.createChar(1, grado);
5     lcd.createChar(2, gota);
6     lcd.createChar(3, reloj);
7
8     dht.begin();
9
10    // Inicializar el RTC
11    if (!rtc.begin()) {
12        lcd.setCursor(0, 0); lcd.print("Error RTC!");
13        while (true) delay(1000);
14    }
15
16    // Si el RTC perdio la hora (pila agotada),
17    // se ajusta a la fecha/hora de compilaci n
18    if (rtc.lostPower()) {
19        rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
20    }
21
22    lcd.setCursor(0, 0);

```

```

23  lcd.print("    FARMACIA");
24  lcd.setCursor(0, 1);
25  lcd.print("    La Chana");
26  delay(1500);
27
28  leerSensor();
29  lcd.clear();
30 }

```

Función loop()

Es el bucle principal que se repite indefinidamente.

Usa `millis()` para llevar varios temporizadores en paralelo sin bloquear el programa:

- Cada 2.5 segundos lee el DHT11.
- Cada 600 milisegundos hace parpadear las cruces.
- Cada 3 segundos cambia el mensaje que se muestra en el centro de la pantalla.

Si el mensaje activo es el reloj, lo refresca cada segundo para que se vean avanzar los segundos.

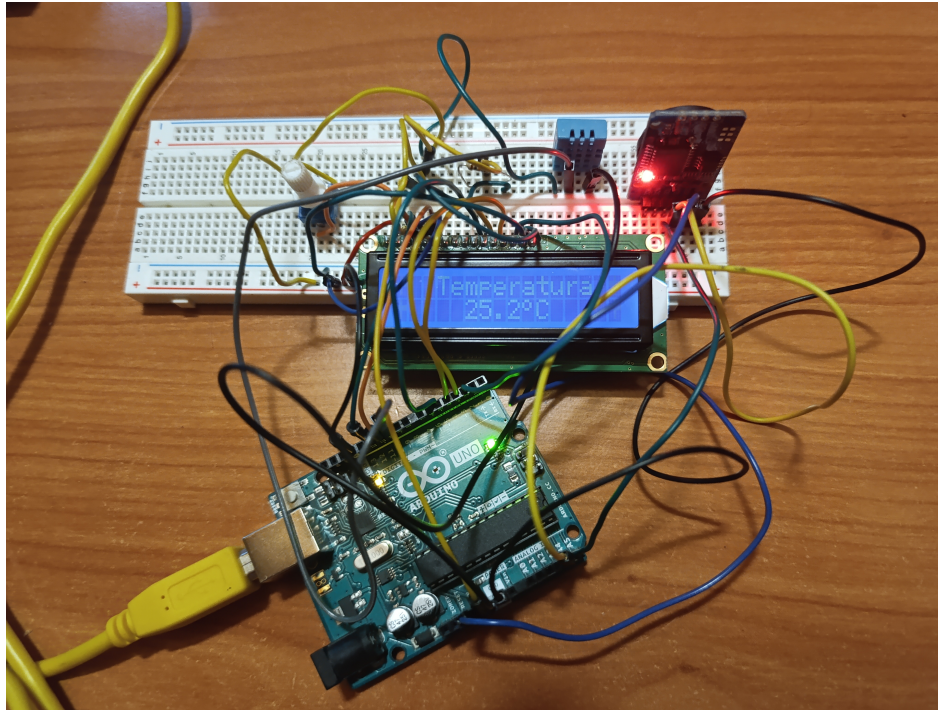
```

1 void loop() {
2   unsigned long ahora = millis();
3
4   // Lectura periodica del DHT11
5   if (ahora - ultimaLecturaDHT >= INTERVALO_LECTURA_DHT) {
6     ultimaLecturaDHT = ahora;
7     leerSensor();
8   }
9
10  // Parpadeo de las cruces
11  if (ahora - ultimoParpadeo >= INTERVALO_PARPADO) {
12    ultimoParpadeo = ahora;
13    cruzVisible = !cruzVisible;
14    dibujarCruces();
15  }
16
17  // Si el mensaje actual es la hora, la refrescamos cada segundo
18  if (mensajeActual == 2 && ahora - ultimaActHora >= INTERVALO_HORA) {
19    ultimaActHora = ahora;
20    mostrarHora();
21  }
22
23  // Cambio de mensaje
24  if (ahora - ultimoCambioMsg >= INTERVALO_MENSAJE) {
25    ultimoCambioMsg = ahora;
26    mensajeActual = (mensajeActual + 1) % 5;
27    mostrarMensaje();
28  }
29 }

```

6. Puesta en escena

El resultado final de este trabajo se muestra en la siguiente imagen.



En la carpeta “Vídeos” del repositorio en Github (<https://github.com/alopejin/PDIH/tree/main/Trabajo/V%C3%ADdeos>) se puede encontrar un vídeo del cartel funcionando.

7. Conclusiones

La principal dificultad haciendo este trabajo ha sido el hecho de no poder simularlo primero en Tinkercad antes de implementarlo físicamente debido a que ni el sensor DHT11 ni el RTC DS3231 estaban disponibles para simular, debido a esto y a mi propia torpeza, durante la implementación provoqué un cortocircuito que fundió uno de los dos sensores DHT11 que tenía (el de mejor calidad por desgracia). Sin embargo el resultado final fue el esperado.

Haber conseguido replicar a pequeña escala un objeto de la vida cotidiana, que a primera vista puede parecer más complejo de lo que en realidad es, me ha ayudado a darme cuenta de que muchas veces nos ponemos barreras nosotros mismos y que las cosas no siempre son tan difíciles como parecen.